PowerShift AI: Orquestador Invisible de Energía y Costos

Definición de proyecto, modelado y arquitectura de la base de datos

Centro universitario de Guadalajara (CUGDL)
Materia: Bases de Datos
Programa Académico: Licenciatura en Inteligencia artificial y Ciencia de datos
Maestra: Nicole Michelle Leyva Munguía

Fecha de Entrega: 17 de Noviembre de 2026

Integrantes del Equipo:

- Gabriel Orozco Alvárez (gabriel.orozco6144@alumnos.udg.mx)
- Yohann Emmanuel Bustos Peñaloza (yohann.bustos1369@alumnos.udg.mx)
- Angel Zoe García Granados (angel.garcia2070@alumnos.udg.mx)
- Juan Pablo Arroyo León (juan.arroyo4946@alumnos.udg.mx)

1. Definición del Proyecto y Descripción de Fuentes de Datos

Definición del Proyecto

PowerShift AI es un orquestador inteligente de energía que opera en la red local (dispositivo Edge / Raspberry Pi) diseñado para optimizar de manera transparente el consumo eléctrico de cargas pesadas en hogares e industria ligera (como aires acondicionados/HVAC, cargadores de vehículos eléctricos, bombas de agua y servidores). El sistema se conectaría en tiempo real a medidores e interruptores IoT a través de protocolos ligeros de red (MQTT/WebSockets), analizaría el comportamiento de las tarifas variables de energía y reprogramaría automáticamente el funcionamiento de estos equipos hacia las horas de menor costo y mayor eficiencia, garantizando en todo momento el confort y las rutinas del usuario.

Alcance de este reporte. Aquí se documenta la capa de datos del sistema y su prototipo. No se cuenta con hardware físico (medidores, interruptores IoT ni Raspberry Pi instalados), por lo que la telemetría proviene de un conjunto de datos público y el comportamiento del optimizador se evalúa en simulación. Lo que sí se implementó es el flujo de datos: ingesta, limpieza, modelo relacional, hypertable de series de tiempo y consultas.

Fuentes de Datos Seleccionadas

Para construir la base de conocimiento y simular el entorno operacional de PowerShift AI se integran dos fuentes de datos complementarias:

1. **Fuente principal: Open Power System Data (OPSD), paquete Household Data, versión 2020-04-15.** Mediciones reales de viviendas y pequeños negocios del sitio piloto de Konstanz, Alemania, con consumo de red, generación fotovoltaica y consumo por aparato. Se usa el archivo de 15 minutos (`household_data_15min_singleindex.csv`, 58 MB): 153,810 filas y 71 columnas, del 11 de diciembre de 2014 al 1 de mayo de 2019, para 11 edificios (6 residenciales, 3 industriales y 2 públicos). Los valores son lecturas **acumuladas** de medidor en kWh. El prototipo usa la vivienda `DE_KN_residential4`, la única residencial que tiene bomba de calor y cargador de vehículo eléctrico medidos a la vez.
2. **Fuente secundaria: CENACE, Sistema de Información del Mercado (SIM).** Precios Marginales Locales del Mercado del Día en Adelanto (MDA), promedios horarios en MXN/MWh, por sistema interconectado (SIN, BCA y BCS) y por hora (1 a 24). El archivo usado (`PMLHorarioMDA.csv`) se descargó del portal público el 2 de noviembre de 2026 y contiene **un solo día** de precios, el 22 de octubre de 2026, sin detalle por nodo.

Sobre Pecan Street Dataport. En entregables anteriores se propuso Pecan Street Dataport como fuente principal. Dataport requiere registrarse y solicitar acceso, y en el formulario de solicitud la Universidad de Guadalajara no aparece entre las instituciones que se pueden elegir, por lo que el equipo no pudo obtener ese acceso para el prototipo. Por eso se trabajó con OPSD, que es de descarga libre. Esto cambia varios campos del modelo (ver la nota de ajuste en la sección 4): no hay circuito de aire acondicionado (la carga térmica es una bomba de calor de calefacción), no hay bomba de agua, y el paquete no trae temperaturas ni ocupación.

Justificación de Uso

Soporte para series de tiempo: la telemetría a 15 minutos (48,576 lecturas limpias solo para la vivienda del prototipo) permite poner a prueba una base de datos en TimescaleDB sobre PostgreSQL, configurando la hypertable, la compresión de datos y las políticas de retención. El volumen del prototipo es de decenas de miles de filas, no de millones; el diseño es el mismo que se usaría con más viviendas o con lecturas por segundo.

Modelado predictivo y bayesiano: el historial por circuito (bomba de calor y vehículo eléctrico) permite entrenar modelos de Machine Learning que predigan la demanda. El conjunto no incluye ocupación ni temperatura, por lo que la probabilidad de ocupación se estimará con una etiqueta aproximada definida en la materia de Machine Learning.

Optimización financiera: cruzar la curva de carga con el PML de CENACE permite calcular el costo de cada hora y plantear un problema de programación lineal que mueva la carga del vehículo eléctrico a las horas más baratas sin tocar la bomba de calor (confort). Como el consumo es de Alemania y los precios son de México, las cifras en pesos ilustran el método y no predicen una factura.

Herramientas y Formatos de Origen de las Fuentes de Datos

Las fuentes de datos seleccionadas emplean las siguientes tecnologías, protocolos y formatos de origen para la extracción e ingesta de datos:

Fuente de Datos	Tipo de Herramienta / Formato	Protocolo de Acceso	Frecuencia de Actualización	Rol en la Arquitectura
OPSD Household Data	Archivo CSV (el paquete también ofrece XLSX y SQLite)	HTTPS, descarga manual en lote	Histórico fijo: 15 min (también hay versiones de 1 y 60 min)	Telemetría residencial del prototipo y entrenamiento de ML.
CENACE SIM	CSV descargado del portal público; servicio web RESTful (JSON) disponible	HTTPS	Horaria (el MDA se publica un día antes)	Insumo de tarifas dinámicas para el optimizador.

2. Arquitectura, Fuentes y Modelado de Datos

Tipo de datos que contiene nuestra fuente

Estructura de la información: toda la información es estructurada: viene en tablas con filas y columnas bien definidas, tanto en el CSV de OPSD como en el reporte de CENACE.

Naturaleza de los datos:

- **Series temporales:** OPSD trae lecturas en marcas de tiempo regulares de 15 minutos (96 por día) y CENACE trae un precio por hora (1 a 24).
- **Datos numéricos continuos:** energía acumulada en kWh (que se convierte a potencia en kW) y PML en MXN/MWh.
- **Datos categóricos y texto:** el edificio y el aparato se identifican por el nombre de la columna (por ejemplo `DE_KN_residential4_ev`); además están el sistema (SIN, BCA, BCS) y el mercado (MDA).

Perfilado de los datos

Antes de modelar se revisó el contenido real de los archivos. Los hallazgos que condicionaron el diseño son:

- **Las lecturas son acumuladas.** En las columnas usadas no hay diferencias negativas (no hay reinicios de medidor), así que al diferenciar se obtiene la energía de cada intervalo y la potencia es kW = 4 × kWh.
- **Cada medidor tiene su propio periodo.** Por eso hay muchos nulos (por ejemplo, `grid_import` tiene 81,435 valores de 153,810 filas). Se usa la ventana en la que existen a la vez la bomba de calor, el vehículo eléctrico y el consumo de red: del 13 de octubre de 2015 al 6 de marzo de 2017.
- **Dos relojes.** `utc_timestamp` está en UTC y `cet_cest_timestamp` está en hora de Europa central con cambio de horario. Se trabaja en hora local (`Europe/Berlin`) y se descartan los 3 días de cambio de horario (con 92 o 100 lecturas) y los 2 días parciales de los extremos de la ventana.
- **Columna `interpolated`.** Es texto con los nombres de las columnas que OPSD interpoló en esa marca de tiempo (121,593 filas no nulas).

- **Resultado de la limpieza:** 511 días en la ventana, 5 descartados, 506 días completos, 48,576 lecturas de 15 minutos y 12,144 horas. No quedaron huecos largos dentro de los días conservados.

La Tabla 1 resume las columnas de la vivienda `DE_KN_residential4`, y la Tabla 2, el PML del archivo de CENACE.

Tabla 1. Columnas de la vivienda DE_KN_residential4 en OPSD (lecturas acumuladas, kWh).

Columna	No nulos	Desde	Hasta	Uso en el prototipo
`grid_import`	81,435	2015-10-10	2018-02-04	Sí (consumo de red)
`grid_export`	81,435	2015-10-10	2018-02-04	Sí (excedente a la red)
`pv`	81,435	2015-10-10	2018-02-04	Sí (generación fotovoltaica)
`heat_pump`	49,239	2015-10-10	2017-03-06	Sí (carga térmica, no se mueve)
`ev`	81,148	2015-10-13	2018-02-04	Sí (carga desplazable)
`freezer`	81,147	2015-10-13	2018-02-04	No
`dishwasher`	50,477	2015-10-13	2017-03-22	No
`washing_machine`	49,028	2015-10-13	2017-03-07	No
`refrigerator`	43,053	2015-10-14	2017-01-04	No

Tabla 2. PML MDA del archivo de CENACE (MXN/MWh, un día, 24 horas).

Sistema	Mínimo (hora)	Máximo (hora)	Promedio	Comentario
SIN	347.46 (hora 11)	2,210.92 (hora 20)	763.07	Pico por la tarde-noche
BCA	329.42 (hora 4)	805.17 (hora 18)	469.45	
BCS	4,892.16 (hora 13)	5,900.01 (hora 21)	5,453.20	Nivel muy distinto al SIN

Borrador e inventario preliminar de campos

Para la parte de la telemetría del consumo eléctrico en el hogar usamos estas columnas del dataset:

- **`utc_timestamp`:** fecha y hora exacta de la lectura, en UTC (formato ISO 8601).
- **`cet_cest_timestamp`:** la misma marca en hora de Europa central, con el desfase del horario de verano.
- **`DE_KN_residential4_grid_import` / `grid_export`:** energía acumulada tomada de la red y entregada a la red, en kWh.
- **`DE_KN_residential4_heat_pump` / `ev` / `pv`:** energía acumulada de la bomba de calor, del cargador del auto eléctrico y de la generación fotovoltaica, en kWh.

Para las tarifas, el archivo de CENACE trae:

- **HORA:** hora del día, de 1 a 24 (la hora 1 es de 00:00 a 01:00).
- **SIN, BCA, BCS:** PML promedio de cada sistema interconectado, en MXN/MWh.

A. Dataset de Telemetría e IoT (Open Power System Data)

Campo / Columna	Tipo de Dato (SQL / Pandas)	Naturaleza	Ejemplo de Valor	Descripción Detallada
utc_timestamp	TIMESTAMPTZ / object (se convierte a datetime64)	Serie Temporal	2014-12-11T17:45:00Z	Marca de tiempo ISO 8601 en UTC de la lectura.
cet_cest_timestamp	VARCHAR / object	Serie Temporal (texto)	2014-12-11T18:45:00+0100	Misma marca en hora de Europa central.
interpolated	TEXT / object	Categórico / Texto	DE_KN_residential3_washing_machine	Columnas que OPSD interpoló en esa marca de tiempo.
DE_KN_residential4_grid_import	NUMERIC / float64	Numérico Continuo (acumulado)	0.013	Energía total tomada de la red, en kWh acumulados.
DE_KN_residential4_heat_pump	NUMERIC / float64	Numérico Continuo (acumulado)	0.010	Energía acumulada de la bomba de calor, en kWh.
DE_KN_residential4_ev	NUMERIC / float64	Numérico Continuo (acumulado)	0.156	Energía acumulada del cargador de vehículo eléctrico, en kWh.
DE_KN_residential4_pv	NUMERIC / float64	Numérico Continuo (acumulado)	0.000	Energía acumulada generada por los paneles, en kWh.

Dataset de Tarifas Eléctricas (CENACE SIM, PML MDA promedios horarios)

Campo / Columna	Tipo de Dato (SQL / Pandas)	Naturaleza	Ejemplo de Valor	Descripción Detallada
HORA	SMALLINT / int64	Numérico Discreto	1	Hora del día de 1 a 24. Se combina con la fecha para formar el inicio de la franja.
SIN	NUMERIC(10,2) / float64	Numérico Continuo	459.23	PML promedio del Sistema Interconectado Nacional, en MXN/MWh.
BCA	NUMERIC(10,2) / float64	Numérico Continuo	372.98	PML promedio del sistema Baja California, en MXN/MWh.
BCS	NUMERIC(10,2) / float64	Numérico Continuo	5561.89	PML promedio del sistema Baja California Sur, en MXN/MWh.

3. Diccionario de datos

El diccionario describe los campos de la base de datos tal como quedan en las seis tablas. Para la telemetría y las tarifas se indica de qué columna de la fuente sale cada campo.

Categoría / Campo	Tipo de dato	Descripción
1. Telemetría de consumo (OPSD → `telemetria_consumo`)		
`id_vivienda`	VARCHAR(30)	Identificador de la vivienda (por ejemplo `DE_KN_residential4`). Relaciona cada lectura con su hogar. Llave foránea a `vivienda`.
`ts`	TIMESTAMPTZ	Inicio del intervalo de 15 minutos, en UTC (viene de `utc_timestamp`). Es la columna de partición de la hypertable.
`grid_import_kw`	NUMERIC(8,4)	Potencia tomada de la red, en kW. Se obtiene diferenciando `grid_import` (kWh acumulados) y multiplicando por 4.
`grid_export_kw`	NUMERIC(8,4)	Potencia entregada a la red, en kW (excedente fotovoltaico).
`pv_kw`	NUMERIC(8,4)	Potencia generada por los paneles fotovoltaicos, en kW.
`heat_pump_kw`	NUMERIC(8,4)	Potencia de la bomba de calor, en kW. Carga térmica que no se reprograma.
`ev_kw`	NUMERIC(8,4)	Potencia del cargador de vehículo eléctrico, en kW. Su carga puede moverse a horas de menor precio.
2. Tarifas (CENACE → `nodo`, `tarifa_pml`)		
`clv_nodo`	VARCHAR(20)	Clave del nodo de precio. En el prototipo representa el promedio de un sistema (`SIN-PROM`, `BCA-PROM`, `BCS-PROM`).
`sistema`	VARCHAR(3)	Sistema interconectado: SIN, BCA o BCS. Vive en `nodo`, no en `tarifa_pml`.
`mercado`	VARCHAR(3)	Tipo de mercado: MDA (Día en Adelanto) o MTR (Tiempo Real). El archivo usado es MDA.
`fecha`	DATE	Día al que aplica el precio.
`hora`	SMALLINT	Hora del día de 1 a 24, como la entrega CENACE (viene de `HORA`).
`pml`	NUMERIC(10,2)	Precio Marginal Local en MXN/MWh. Insumo principal del optimizador.
`pml_ene`, `pml_per`, `pml_cng`	NUMERIC(10,2)	Componentes de energía, pérdidas y congestión del PML. Quedan en NULL porque el archivo usado solo trae el PML total.
3. Catálogo y orquestación (generados por el sistema)		
`zona_horaria`	VARCHAR(40)	Zona horaria de la vivienda (`Europe/Berlin`); sirve para convertir `ts` a hora local en el join.
`fuente`	VARCHAR(60)	Origen de los datos de la vivienda (OPSD Household Data, versión 2020-04-15).
`id_dispositivo`	SERIAL	Identificador del equipo controlado.

Categoría / Campo	Tipo de dato	Descripción
`tipo_carga`	VARCHAR(20)	Categoría del equipo: HVAC, HEAT_PUMP, EV, BOMBA o SERVIDOR. En el prototipo hay HEAT_PUMP y EV.
`columna_origen`	VARCHAR(60)	Columna de OPSD de la que sale el consumo del equipo.
`desplazable`	BOOLEAN	TRUE si la carga puede moverse de horario sin afectar el confort (EV); FALSE si debe operar sin interrupción (bomba de calor).
`encendido`	BOOLEAN	Estado actual del equipo. Queda en FALSE por defecto: no hay interruptores IoT reales.
`id_programacion`	SERIAL	Llave primaria de cada decisión del optimizador.
`inicio_programado`	TIMESTAMPTZ	Inicio de la hora en que el optimizador asignó la carga del equipo.
`energia_kwh`	NUMERIC(8,4)	Energía asignada a esa hora, en kWh.
`costo_estimado`	NUMERIC(10,4)	Costo en MXN: energía (kWh) × PML ÷ 1000, porque el PML viene en MXN/MWh.
`prob_ocupacion`	NUMERIC(4,3)	Probabilidad (0 a 1) de que el usuario esté en casa. Queda en NULL hasta contar con el modelo de ocupación de la materia de ML.

4. Estructura relacional y cardinalidad del proyecto

Definición de relaciones entre fuentes y tablas

En PowerShift AI no trabajamos con una sola tabla: consumimos dos fuentes externas que no se conocen entre sí, la telemetría de consumo de la vivienda (OPSD) y los precios de la red eléctrica (CENACE SIM), y además generamos una tercera parte con las tablas de orquestación, que es donde el sistema decide a qué hora operar cada equipo. El modelo queda en seis tablas conectadas entre sí: `nodo`, `vivienda`, `telemetria_consumo`, `tarifa_pml`, `dispositivo` y `programacion` (ver el diagrama entidad-relación más abajo).

Tres de ellas (`nodo`, `vivienda` y `programacion`) no corresponden a columnas de las fuentes: las derivamos del diseño del modelo. `nodo` y `vivienda` sirven para ligar cada casa con su punto de precio de la red, y `programacion` guarda el resultado del optimizador, con `id_programacion` como llave primaria. `dispositivo` tampoco viene en las fuentes: se arma a partir de las columnas de OPSD (`heat_pump` y `ev`).

La conexión entre las tablas funciona de dos formas distintas, y conviene explicarlas por separado porque no son lo mismo:

- **Relaciones por llave foránea (dentro de nuestra base):** `clv_nodo` conecta cada vivienda y cada precio con su nodo; `id_vivienda` conecta la vivienda con sus lecturas y con sus dispositivos; e `id_dispositivo` conecta cada equipo con sus programaciones.
- **Relación temporal (entre las dos fuentes externas):** la telemetría y las tarifas no comparten ninguna llave; lo único que tienen en común es la hora y el nodo. Se cruzan con un join por hora (la hora local de la vivienda más uno, porque CENACE numera las horas de 1 a 24, contra `hora` de `tarifa_pml`) y por nodo, que se obtiene pasando por `vivienda`, porque ahí

vive `clv_nodo`. Este cruce permite calcular el costo: potencia (kW) × horas × PML ÷ 1000, ya que el PML viene en MXN/MWh.

No pusimos una llave foránea directa entre la telemetría y las tarifas a propósito. Son fuentes independientes con frecuencias distintas (lecturas cada 15 minutos contra precios por hora) y además cubren periodos distintos: la telemetría es de 2015 a 2017 y el PML disponible es de un solo día. Forzar una llave rompería la ingesta, así que el cruce se hace en las consultas y no en el esquema. Para poder cruzarlos se creó la vista `v_perfil_pml`, que promedia el PML por nodo, mercado y hora: con un día cargado equivale a ese día y con muchos días funciona como perfil horario típico. Si se dispusiera de PML de las mismas fechas que la telemetría, bastaría agregar `fecha` al cruce.

El campo `sistema` (SIN, BCA o BCS) lo dejamos en la tabla `nodo` y no en `tarifa_pml`, porque el sistema depende del nodo y no de cada precio; así evitamos repetirlo en cada fila de precios. El archivo de CENACE usado trae el PML promedio por sistema, no por nodo, por lo que cada registro de `nodo` representa el promedio de un sistema (`SIN-PROM`, `BCA-PROM`, `BCS-PROM`) y la vivienda se asigna a `SIN-PROM` como supuesto del escenario. Si se descargara el PML por nodo, solo habría que agregar filas a `nodo` y `tarifa_pml`.

Tipo de cardinalidad

Relación	Cardinalidad	Justificación
nodo → *vivienda*	1:N	Varias viviendas pueden estar en el mismo nodo de la red, pero cada vivienda cae en un solo nodo (`clv_nodo`).
nodo → *tarifa_pml*	1:N	Cada nodo tiene un precio distinto por cada hora, día y mercado (MDA o MTR), entonces acumula muchos registros.
vivienda → *telemetria_consumo*	1:N	Una casa genera miles de lecturas a lo largo del tiempo, pero cada lectura pertenece a una sola casa (`id_vivienda`).
vivienda → *dispositivo*	1:N	Una casa tiene varios equipos controlables (bomba de calor, EV), pero cada equipo está instalado en una sola casa.
dispositivo → *programacion*	1:N	Un equipo se reprograma muchas veces (cada día, cada hora de carga), pero cada programación es de un solo equipo.
telemetria_consumo → *tarifa_pml*	N:1 (por join temporal)	Muchas lecturas de una misma hora y nodo comparten el mismo precio, pero cada lectura corresponde a un solo precio por mercado.
dispositivo ↔ *tarifa_pml* (franjas horarias)	N:M	Un equipo puede ser movido a muchas horas y una misma hora puede tener muchos equipos programados; por eso `programacion` funciona como tabla intermedia y guarda `inicio_programado`, `energia_kwh`, `costo_estimado` y `prob_ocupacion`.

La única relación N:M real es la de dispositivos con horas de menor costo, y se resuelve con la tabla `programacion`, que es justo el resultado del optimizador (guarda qué equipo, a qué hora y con qué costo). Todo lo demás es 1:N porque así funcionan las jerarquías normales de este negocio: una zona tiene muchas casas, una casa tiene muchas lecturas y muchos equipos, pero nunca al revés.

Diagrama entidad-relación

La Figura 1 muestra el modelo con los campos reales de cada tabla.

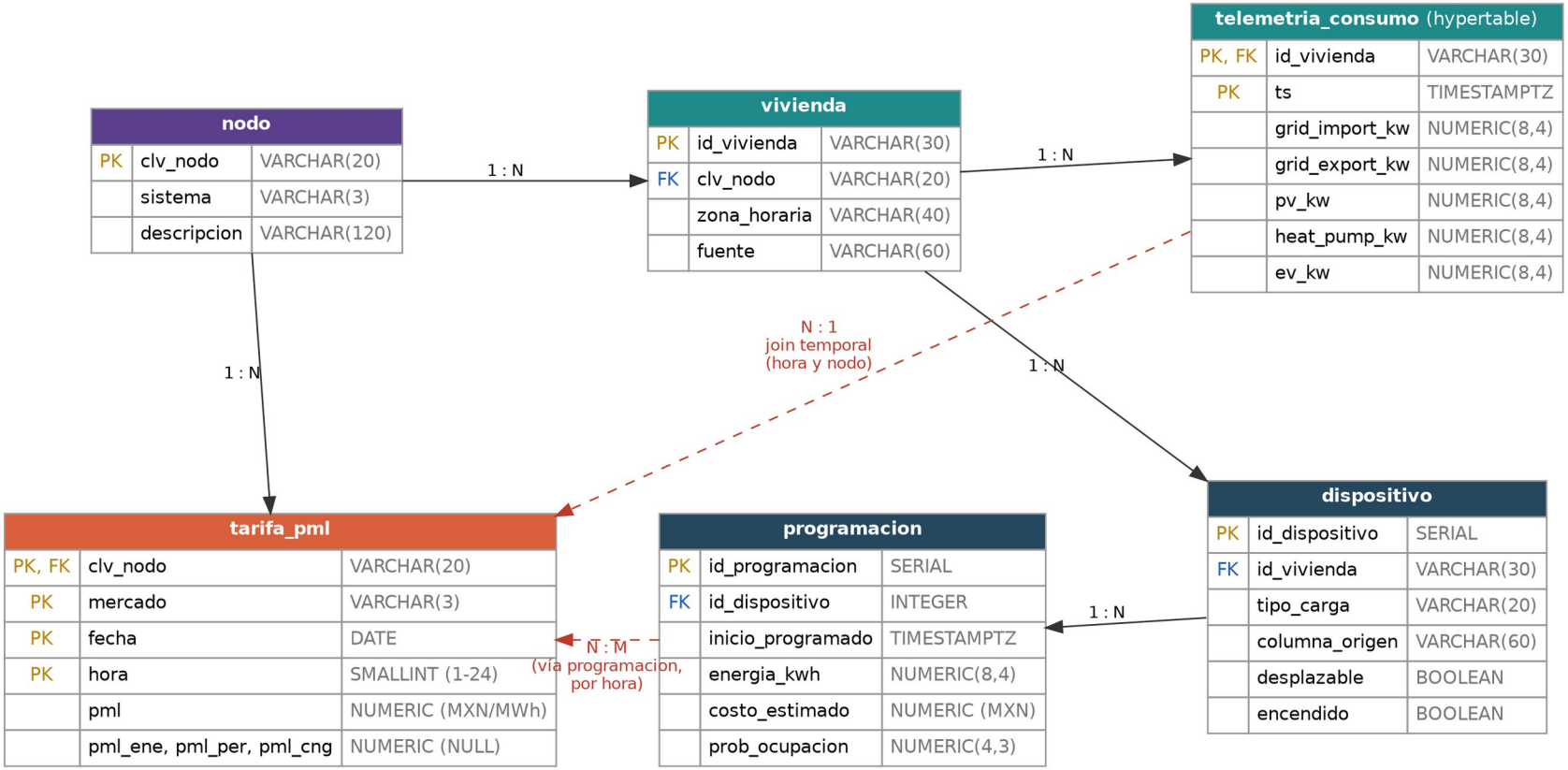


Figura 1. Modelo entidad-relación de PowerShift AI. Las líneas continuas son llaves foráneas y las punteadas son relaciones temporales que se resuelven en las consultas.

Nota de ajuste respecto al entregable anterior

Las entregas anteriores describían a Pecan Street Dataport como fuente principal. Al trabajar con los archivos reales que sí se pudieron descargar, el modelo cambió en lo siguiente (las seis tablas y sus relaciones se mantienen):

Elemento	Antes (Pecan Street)	Ahora (OPSD y CENACE)
Telemetría	Pecan Street Dataport, 1 s / 1 min	OPSD Household Data, 15 min
Identificador de vivienda	`dataid` (entero)	`id_vivienda` (texto, p. ej. `DE_KN_residential4`)
Marca de tiempo	`localminute`	`ts` (UTC, de `utc_timestamp`)
Consumo de red	`grid` (neto, kW)	`grid_import_kw` y `grid_export_kw` (de lecturas acumuladas)
Solar	`solar`	`pv_kw`
Climatización	`air1` (aire acondicionado)	`heat_pump_kw` (bomba de calor de calefacción)
Vehículo eléctrico	`car1`	`ev_kw`
Bomba de agua / refrigerador	`wellpump1`, `refrigerator1`	No se ingieren (no existe / cobertura limitada)
Temperaturas	`temperatura_exterior`, `temperatura_interior`	No vienen en este paquete
Precio	PML por nodo, con componentes y factor de emisión	PML total promedio por sistema (componentes en NULL)
Marca de tiempo del precio	`fecha_hora`	`fecha` y `hora` (1 a 24), como las entrega CENACE

El PML se sigue manejando en MXN/MWh y por eso el costo estimado se divide entre 1000.

5. Arquitectura del dato

Diagrama de la arquitectura del dato

En la Figura 2 las cajas con borde sólido son lo que se implementó en el prototipo; las de borde punteado son diseño o forman parte de otra materia.

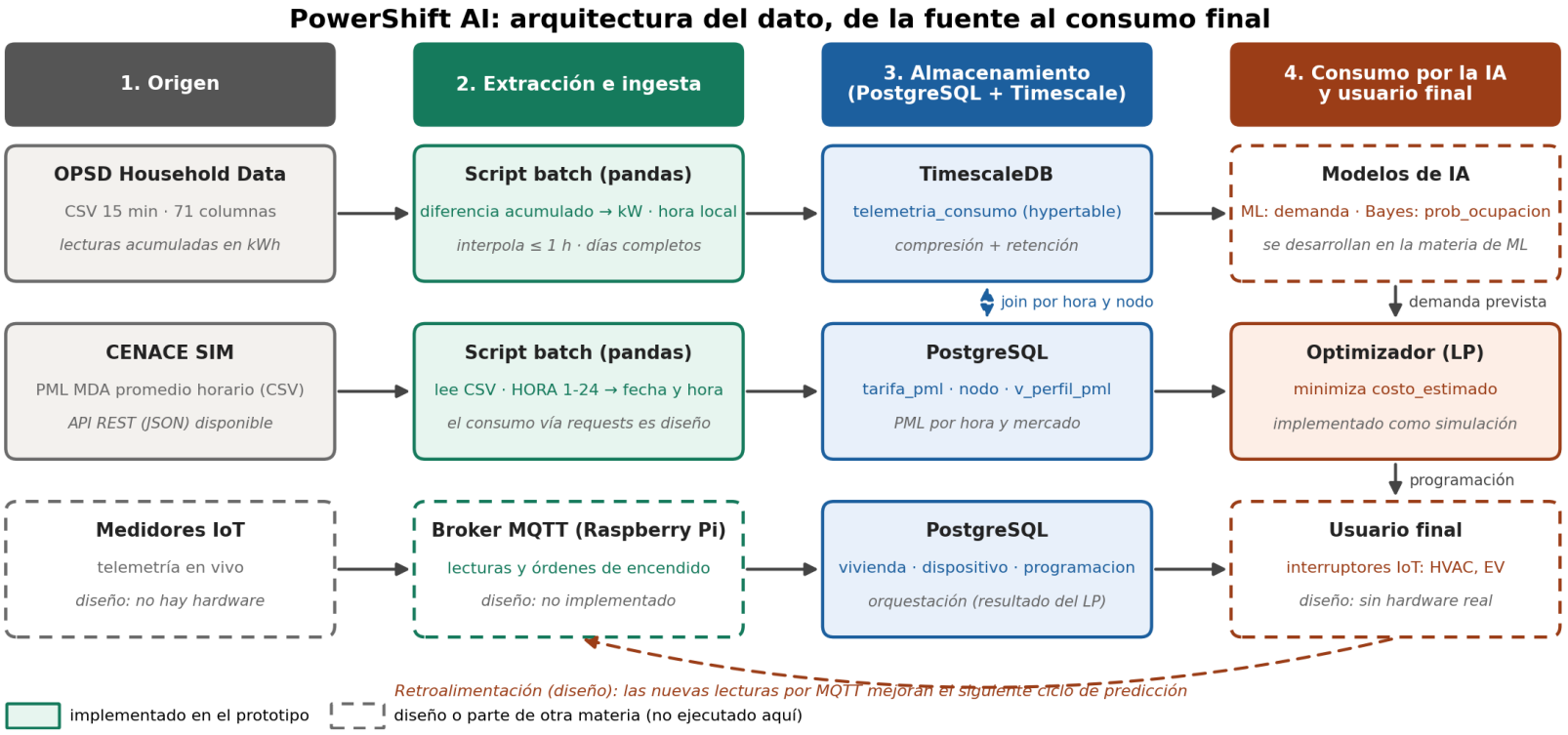


Figura 2. Arquitectura del dato de PowerShift AI, de la fuente al consumo final.

Explicación narrativa del diagrama

El flujo de PowerShift AI sigue el camino del dato en cuatro etapas: origen, extracción e ingesta, almacenamiento y consumo final. Tres líneas de datos corren en paralelo y se juntan hasta la etapa de consumo, donde el optimizador toma la decisión de a qué hora cargar cada equipo.

1. Origen del dato

Tenemos tres orígenes. Dos son fuentes externas: OPSD Household Data (telemetría por circuito en CSV) y CENACE SIM (precios PML en CSV, o en JSON por servicio web). El tercero son los medidores e interruptores IoT de la vivienda, que mandarían lecturas en tiempo real; no tenemos hardware físico instalado, por lo que en el prototipo la telemetría sale del histórico de OPSD.

2. Extracción e ingesta

Cada origen entra por su propio camino, porque cada uno trae un formato y una frecuencia distintos:

- **Script batch (Python + pandas) para OPSD:** lee el CSV, selecciona la vivienda, **diferencia las lecturas acumuladas** para obtener kWh por intervalo, convierte a kW, pasa de UTC a hora local, interpola huecos de hasta una hora, descarta los días incompletos (quedan 506) y genera `telemetria_15min.csv`.
- **Script batch (Python + pandas) para CENACE:** lee el CSV (con un encabezado de 4 líneas), convierte `HORA` y arma un registro por sistema y por hora. La consulta periódica a la API con `requests` (GET y JSON) queda como diseño para cuando se necesiten más días.
- **Broker MQTT en la Raspberry Pi (Edge):** diseño no implementado. Recibiría las lecturas de los medidores IoT con protocolos ligeros y sería el mismo canal por el que regresarían las órdenes de encendido y apagado.

En esta etapa también se limpia lo que llega: se revisan nulos, se unifican las unidades (kW para potencia y MXN/MWh para PML) y se descartan los días incompletos.

3. Almacenamiento y procesamiento local

Todo se guarda en PostgreSQL con la extensión TimescaleDB, dentro de las seis tablas del modelo relacional:

- `telemetria_consumo` es la hypertable, particionada por `ts`. Aquí se aplican la compresión y la política de retención, porque es la tabla que más crece.
- `tarifa_pml` y `nodo` guardan los precios por hora y mercado, y el sistema (SIN, BCA o BCS) vive en `nodo` para no repetirlo en cada precio.
- `vivienda`, `dispositivo` y `programacion` guardan la parte de orquestación. `programacion` es el resultado del optimizador.

El procesamiento ocurre en las consultas. La telemetría y las tarifas no tienen llave foránea entre sí, así que se cruzan con un join temporal por hora (hora local de `ts` más uno) y por nodo, pasando por `vivienda`. Con ese cruce se calcula el costo como kW × horas × PML ÷ 1000.

4. Consumo por la IA y el usuario final

La información ya integrada alimenta tres componentes:

- **Modelos de IA:** el modelo de Machine Learning que predice la demanda y la inferencia bayesiana que calcula `prob_ocupacion` se desarrollan en la materia de Machine Learning; aquí solo se deja preparada la tabla.
- **Optimizador (Programación Lineal):** toma los precios PML y reparte la energía diaria del vehículo eléctrico en las horas permitidas (de 18:00 a 08:00) minimizando el costo, con una potencia máxima de 3.2 kW. La bomba de calor no se mueve, para proteger el confort. Se implementó como simulación. La ventana de 18:00 a 08:00 es un supuesto de diseño y no un hábito observado (ver las limitaciones).
- **Usuario final:** el resultado se guarda en `programacion` y se enviaría por MQTT a los interruptores IoT. Esa parte es diseño, porque no hay hardware real.

El sistema se retroalimentaría: las nuevas lecturas de los medidores volverían a entrar por el broker MQTT y mejorarían las predicciones del siguiente ciclo (diseño).

Implementación en PostgreSQL y TimescaleDB

La Tabla 3 separa lo que se implementó y ejecutó en el prototipo de lo que queda como diseño.

Tabla 3. Alcance del prototipo.

Componente	Estado	Dónde está
Ingesta y limpieza de OPSD y CENACE (pandas)	Implementado	Notebook `00_ingesta_limpieza`
Esquema de 6 tablas, llaves y restricciones	Implementado	`esquema_powershift.sql`
Hypertable, compresión y política de retención	Implementado	`esquema_powershift.sql` y notebook `01_bases_de_datos`
Carga de datos, join temporal y consultas	Implementado	Notebook `01_bases_de_datos` y `consultas_powershift.sql`
Optimizador lineal (simulación)	Implementado como simulación	Notebook `04_optimizador_lp`
Modelos de ML y de ocupación	Otra materia	Reporte de Machine Learning
Broker MQTT, medidores e interruptores IoT	Solo diseño	No hay hardware físico

Esquema y llaves. El script `esquema_powershift.sql` crea las seis tablas con llaves primarias, llaves foráneas y restricciones `CHECK` (por ejemplo, `hora` entre 1 y 24 y `sistema` en SIN, BCA o BCS). La llave primaria de `telemetria_consumo` es `(id_vivienda, ts)`, porque TimescaleDB exige que la columna de partición forme parte de ella.

Hypertable. `telemetria_consumo` se convierte en hypertable con `create_hypertable` particionada por `ts` en chunks de 30 días. Con el periodo cargado (del 13 de octubre de 2015 al 5 de marzo de 2017 en UTC) resultan 18 chunks, y eso fue lo que se obtuvo al ejecutar el notebook en PostgreSQL 16 con TimescaleDB 2.31.0.

Compresión. Se habilita la compresión por columnas, segmentada por `id_vivienda` y ordenada por `ts` descendente, con una política que comprime los chunks de más de 90 días. En el notebook se comprimen los chunks manualmente para medir el tamaño antes y después: según

`hypertable_compression_stats`, los 18 chunks pasaron de 11,304,960 bytes (11.3 MB) a 1,966,080 bytes (2.0 MB), una reducción de 82.6 %, es decir, 5.7 veces menos espacio. TimescaleDB programó además los trabajos en segundo plano: la compresión se evalúa cada 12 horas y la retención cada día.

Retención. La política de retención se fijó en 20 años. Es un valor deliberadamente largo: los datos de OPSD son de 2015 a 2017, y un plazo corto borraría todo el histórico del prototipo. En una instalación real, con datos nuevos, se usaría un plazo corto (por ejemplo, dos años de lecturas de 15 minutos) y se conservarían agregados por hora.

Carga. Los catálogos (`nodo`, `vivienda`, `dispositivo`) se insertan directamente y la telemetría, las tarifas y la programación se cargan con `COPY` desde los archivos que generan los notebooks 00 y 04. El contenido cargado es de 3 nodos, 1 vivienda, 2 dispositivos, 72 tarifas (3 sistemas × 24 horas), 48,576 lecturas de telemetría y 572 programaciones.

Join temporal y validación. La consulta Q1 cruza `telemetria_consumo` con `v_perfil_pml` por hora y por nodo y calcula el costo diario; Q2 compara el costo sin orquestador contra el costo con el horario de `programacion`, y Q3 muestra a qué horas programó el optimizador cada equipo. El notebook compara los costos que devuelve SQL contra los que calculó pandas en el notebook 04, con una tolerancia de 0.01 MXN (las columnas son `NUMERIC(8,4)` y se redondea al guardar). En la ejecución, las diferencias máximas sobre los 268 días fueron de 5.0×10^{-5} MXN en el costo sin orquestador y de 1.1×10^{-4} MXN en el costo con orquestador, por lo que la base de datos reproduce los resultados del notebook 04.

Resultado de la simulación. Sobre los 268 días con carga de vehículo eléctrico, el costo medio diario de la bomba de calor y el EV es de 11.06 MXN sin orquestador y 8.83 MXN con el horario del optimizador: un ahorro simulado de 2.23 MXN por día, equivalente a 20.2 % del costo controlable y 54.3 % del costo del EV. El optimizador concentró la carga en la madrugada (Tabla 4), pero la ventana de 18:00 a 08:00 es un supuesto: según el análisis del reporte de Machine Learning, en los datos observados solo el 7.2 % de la energía del EV se carga en ese horario y el 83.0 % se carga entre las 10:00 y las 17:00. Ese mismo análisis encuentra que el ahorro simulado casi no cambia con una ventana aprendida del hábito (alrededor de 54 % del costo del EV). Son cifras de una simulación con consumo de Alemania y precios de México, y por eso no representan una factura real.

Tabla 4. Horas en que el optimizador programó la carga del EV (tabla programacion).

Hora local	Programaciones	Energía (kWh)	PML promedio (MXN/MWh)
02:00	82	160.7	363.34
03:00	200	445.2	357.08
04:00	268	780.1	355.16
05:00	20	22.6	365.88
07:00	2	2.5	371.47

6. Conclusiones y bibliografía

Conclusión: lo que nos llevamos del proyecto (Reflexión)

- **¿Por qué importa tener los datos bien estructurados para la IA?** Porque un modelo es tan bueno como los datos que recibe y revisar los archivos reales nos lo demostró, ya que si hubiéramos tomado las columnas de OPSD tal cual habríamos tratado kWh acumulados como si fueran potencia, además de que hubo que resolver dos relojes distintos entre UTC y hora local, los días de cambio de horario, huecos de datos y un PML con horas numeradas de 1 a 24, todo lo cual nos deja ver que tener el modelo y la limpieza definidos de antemano permite que los algoritmos de ML y el optimizador trabajen sobre datos coherentes, logrando que las decisiones se puedan explicar y reproducir.
- **¿De qué nos sirvió la materia de Bases de Datos para Ciencia de Datos e IA?** Nos dimos cuenta de que una base de datos no es solo el lugar donde se guardan las cosas, porque con una hypertable de TimescaleDB, compresión y un join temporal por hora y por nodo el trabajo pesado de agrupar y calcular costos se le deja al motor de SQL, y comparar sus resultados contra los de pandas nos sirvió para validar el diseño, dado que nosotros aunque seguimos en formación vamos a tener ese papel como científicos o analistas de datos, esa parte nos ayuda bastante porque también aprendimos a decidir qué va en el esquema y qué va en las consultas, viendo que la relación entre telemetría y tarifas no lleva llave foránea justamente porque las fuentes son independientes.

Limitaciones

- Los datos de consumo son de viviendas de Alemania y los precios son de México; las cifras en pesos ilustran el método.
- El archivo de CENACE contiene un solo día de PML promedio por sistema, no una serie histórica ni precios por nodo; se usa como perfil horario típico.
- No hay hardware real: el broker MQTT, los medidores y los interruptores IoT son diseño, y el ahorro es una simulación.
- La ventana de carga nocturna (18:00 a 08:00) del optimizador es un supuesto de diseño: en los datos observados, la carga del EV ocurre sobre todo entre las 10:00 y las 17:00.
- El paquete OPSD no incluye temperaturas ni ocupación, por lo que esos campos quedan pendientes (la ocupación se aproximará en la materia de ML).

Referencias

Centro Nacional de Control de Energía. (2026). *Sistema de Información del Mercado (SIM): Precios Marginales Locales, MDA, promedios horarios* [Conjunto de datos]. CENACE. https://www.cenace.gob.mx/APSIM.aspx (PML del 22 de octubre de 2026; consultado el 2 de noviembre de 2026).

McKinney, W. (2010). Data structures for statistical computing in Python. En *Proceedings of the 9th Python in Science Conference* (pp. 56-61).

Open Power System Data. (2020). *Data package household data* (Versión 2020-04-15) [Conjunto de datos]. https://data.open-power-system-data.org/household_data/2020-04-15/

Pecan Street Inc. (s. f.). *Dataport: Residential energy & smart meter telemetry*. https://www.pecanstreet.org/dataport/ (fuente considerada y descartada por falta de acceso).

PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Timescale, Inc. (2026). *TimescaleDB documentation*. https://docs.timescale.com/

Uso de IA en la elaboración del documento

Declaración de transparencia: para este proyecto usamos un asistente de inteligencia artificial generativa (Claude, de Anthropic) como apoyo, lo usamos para revisar la estructura de los archivos descargados, proponer el esquema y las consultas SQL, escribir el código de limpieza y de carga de los notebooks y redactar versiones del texto, también nos orientó sobre los pasos que convenía seguir, sobre cómo estructurar el documento y sobre los formatos que ayudaban a pulir la presentación, y el equipo revisó el contenido, ejecutó el código y es responsable de las decisiones de diseño, de los resultados y de lo que aquí se entrega.

Anexo de evidencias del notebook

A. Lectura y perfilado inicial del dataset (OPSD)

```
import pandas as pd

df = pd.read_csv('household_data_15min_singleindex.csv')
print(df.shape)
print(df.info())
```

Salida (fragmento):

```
(153810, 71)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 153810 entries, 0 to 153809
Data columns (total 71 columns):
 0  utc_timestamp                  153810 non-null  object
 1  cet_cest_timestamp              153810 non-null  object
 2  DE_KN_industrial1_grid_import    65730 non-null  float64
 ...
```

B. Limpieza y construcción de la telemetría (notebook 00)

```
raw = raw[[f"{VIVIENDA}_{c}" for c in CARGAS]]      # CARGAS = grid_import, heat_pump, ev
dif = raw.diff()                    # lecturas acumuladas -> kWh por intervalo de 15 min
kw = dif * 4                        # kW = kWh / 0.25 h
kw.index = kw.index.tz_convert("Europe/Berlin")
kw = kw.interpolate(method="time", limit=4)          # huecos de hasta 1 h
dias_ok = n_filas[(n_filas == 96) & (n_nulos == 0)].index
```

Salida:

```
Diferencias negativas (reinicios de medidor): 0 en grid_import, heat_pump y ev
Ventana con las 3 cargas: 2015-10-13 16:15 UTC -> 2017-03-06 14:00 UTC
Días en la ventana: 511
Días descartados por no tener 96 lecturas (3 cambios de horario + 2 días parciales): 5
Días con huecos largos, descartados: 0
```

```
Días completos conservados: 506
telemetria_15min.csv: 48,576 filas | datos_horarios.csv: 12,144 filas
```

C. Esquema: hypertable, compresión y retención (fragmento de esquema_powershift.sql)

```
SELECT create_hypertable('telemetria_consumo', 'ts',
                         chunk_time_interval => INTERVAL '30 days', if_not_exists => TRUE);

ALTER TABLE telemetria_consumo SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'id_vivienda',
    timescaledb.compress_orderby   = 'ts DESC');
SELECT add_compression_policy('telemetria_consumo', INTERVAL '90 days');
SELECT add_retention_policy('telemetria_consumo', INTERVAL '20 years');
```

D. Join temporal telemetría ↔ tarifas (consulta Q1)

```
SELECT t.id_vivienda,
       (t.ts AT TIME ZONE v.zona_horaria)::date AS fecha_local,
       ROUND(SUM((t.heat_pump_kw + t.ev_kw) * 0.25 * p.pml / 1000.0), 4) AS
costo_sin_orquestador_mxn
FROM telemetria_consumo t
JOIN vivienda     v ON v.id_vivienda = t.id_vivienda
JOIN v_perfil_pml p ON p.clv_nodo = v.clv_nodo AND p.mercado = 'MDA'
                   AND p.hora = EXTRACT(HOUR FROM (t.ts AT TIME ZONE v.zona_horaria))::int + 1
GROUP BY t.id_vivienda, fecha_local ORDER BY fecha_local LIMIT 10;
```

Las consultas Q2 (ahorro simulado) y Q3 (horas programadas) están completas en `consultas_powershift.sql`.

E. Evidencia de la ejecución en PostgreSQL con TimescaleDB

Salidas obtenidas al ejecutar el notebook `01_bases_de_datos` en Google Colab, con PostgreSQL 16 y TimescaleDB 2.31.0 instalados en la sesión. Las celdas corrieron en orden y sin errores.

E.1 Extensión, tablas creadas y filas cargadas

```
       extname extversion
0  timescaledb     2.31.0
          table_name
0         dispositivo
1               nodo
2        programacion
3          tarifa_pml
4  telemetria_consumo
5        v_perfil_pml
6            vivienda

nodo                        3 filas
vivienda                    1 filas
tarifa_pml                 72 filas
dispositivo                 2 filas
telemetria_consumo     48,576 filas
programacion              572 filas
```

E.2 Hypertable, chunks y políticas

```
      hypertable_name  num_chunks  compression_enabled
0  telemetria_consumo          18                 True
            proc_name     hypertable_name schedule_interval
0  policy_compression  telemetria_consumo   0 days 12:00:00
1    policy_retention  telemetria_consumo   1 days 00:00:00
```

E.3 Compresión: tamaño antes y después

```
  tamano_antes tamano_despues
0        11 MB        2224 kB
                                      0
total_chunks                         18
number_compressed_chunks             18
before_compression_table_bytes   6291456
before_compression_index_bytes   5013504
before_compression_toast_bytes         0
before_compression_total_bytes  11304960
after_compression_table_bytes     294912
after_compression_index_bytes     294912
after_compression_toast_bytes    1376256
after_compression_total_bytes    1966080
node_name                           None
```

E.4 Join temporal telemetría ↔ tarifas (Q1)

```
          id_vivienda fecha_local costo_sin_orquestador_mxn
0  DE_KN_residential4  2015-10-14                     9.9758
1  DE_KN_residential4  2015-10-15                     6.2090
2  DE_KN_residential4  2015-10-16                    13.0184
3  DE_KN_residential4  2015-10-17                    10.5341
4  DE_KN_residential4  2015-10-18                    17.4774
5  DE_KN_residential4  2015-10-19                     5.7227
6  DE_KN_residential4  2015-10-20                     5.6863
7  DE_KN_residential4  2015-10-21                     3.4372
8  DE_KN_residential4  2015-10-22                     8.9860
9  DE_KN_residential4  2015-10-23                     5.0040
```

E.5 Ahorro simulado (Q2) y horas programadas por el optimizador (Q3)

```
Días con carga de EV en la consulta: 268
  fecha_local costo_sin costo_con  ahorro
0  2015-10-14    9.9758    6.2626  3.7131
1  2015-10-16   13.0184    9.4159  3.6025
2  2015-10-17   10.5341    8.7548  1.7793
3  2015-10-18   17.4774   12.4684  5.0090
4  2015-10-20    5.6863    3.9877  1.6986
  tipo_carga  hora_local  n_programaciones     kwh pml_mxn_mwh
0         EV           2                82  160.71      363.34
1         EV           3               200  445.24      357.08
2         EV           4               268  780.13      355.16
3         EV           5                20   22.59      365.88
4         EV           7                 2    2.54      371.47
```

E.6 Validación cruzada contra pandas

```
Días comparados: 268
Máx. diferencia costo_sin (MXN): 4.9679999740703806e-05
Máx. diferencia costo_con (MXN): 0.00011314999975198248
Ahorro medio diario: 2.2285 MXN | costo medio sin: 11.059 | con: 8.830
OK: la base de datos reproduce los resultados del notebook 04.
```